

The Fourth Thief: Your Own Locks

Lab course • Lecture 1 • Synchronization primitives in C++ and Java

Three thieves down. The fourth one is you

The first three stole your speedup from outside. This one you wrote yourself.



Where we are going

- races, atomics, and one measurement;
- the mutex: what it costs, how coarse to make it;
- the memory model: when you see another write;
- waiting: condvars, semaphores, barriers;
- guarantees: fairness, starvation, philosophers;
- and the tools that catch what you missed.

The ladder, top to bottom

share nothing → immutable data → a ready-made concurrent container → atomic → mutex → **your own lock-free structure**.

Take the highest rung that does the job. The next hour is about what each rung actually promises.

? QUESTION FOR YOU

Amdahl caps you at $1/(1 - p)$, with $1 - p$ the serial fraction. Where in that formula does your `lock()` live?

Three thieves down. The fourth one is you

The first three stole your speedup from outside. This one you wrote yourself.



Where we are going

- races, atomics, and one measurement;
- the mutex: what it costs, how coarse to make it;
- the memory model: when you see another write;
- waiting: condvars, semaphores, barriers;
- guarantees: fairness, starvation, philosophers;
- and the tools that catch what you missed.

The ladder, top to bottom

share nothing → immutable data → a ready-made concurrent container → atomic → mutex → **your own lock-free structure**.

Take the highest rung that does the job. The next hour is about what each rung actually promises.

? QUESTION FOR YOU

Amdahl caps you at $1/(1 - p)$, with $1 - p$ the serial fraction. Where in that formula does your `lock()` live?

→ ANSWER

Right inside $1 - p$. A critical section is serial by definition — every lock you take lowers your own ceiling.

Data race $\neq$ race condition

Two bugs everyone mixes up. One is broken memory, the other is broken logic — and each needs a different fix.

Data race — a bug in memory

One location, two threads, at least one write, zero synchronization.

```
int counter = 0;
void worker() {
    for (int i=0; i<1000000; i++)
        counter++; // data race
}
```

C/C++: plain undefined behavior.

Java: not UB — reads and writes of an `int` are atomic — but updates are silently lost.

Race condition — a bug in logic

The outcome depends on who got there first. Happens without a data race too.

```
// map = ConcurrentHashMap
if (!map.containsKey(k)) // check
    map.put(k, v);        // then act
// do this instead: map.putIfAbsent(k, v);
```

Every call here is atomic on its own, and the map is thread-safe. The pair of them is not.

? QUESTION FOR YOU

We make every field atomic. Do the races go away?

Data race $\neq$ race condition

Two bugs everyone mixes up. One is broken memory, the other is broken logic — and each needs a different fix.

Data race — a bug in memory

One location, two threads, at least one write, zero synchronization.

```
int counter = 0;
void worker() {
    for (int i=0; i<1000000; i++)
        counter++; // data race
}
```

C/C++: plain undefined behavior.

Java: not UB — reads and writes of an `int` are atomic — but updates are silently lost.

Race condition — a bug in logic

The outcome depends on who got there first. Happens without a data race too.

```
// map = ConcurrentHashMap
if (!map.containsKey(k)) // check
    map.put(k, v);        // then act
// do this instead: map.putIfAbsent(k, v);
```

Every call here is atomic on its own, and the map is thread-safe. The pair of them is not.

? QUESTION FOR YOU

We make every field atomic. Do the races go away?

→ ANSWER

The **data race** does. The **race condition** stays: atomic steps never add up to an atomic sequence. Remember this one — it comes back at the end of the lecture.

Atomics: the cheapest synchronization there is

The hardware does read–modify–write as one indivisible step, and never enters the kernel. It is cheap — but it is not free.

Ready-made atomic operations

```
std::atomic<int> counter{0};  
counter.fetch_add(1); // one locked RMW
```

```
AtomicInteger c = new AtomicInteger();  
c.incrementAndGet();  
LongAdder a = new LongAdder(); // sharded
```

LongAdder grows *extra cells* under contention and sums them on read — that is the whole trick, and we will measure it in a minute.

What you actually pay for

To modify a value, a core must first own its **cache line** exclusively — it takes the line away from every other core.

One core doing that: pennies.

Sixteen cores doing that to the *same* line: the line ping-pongs between them, and your parallel program spends its life moving 64 bytes around.

■ RULE OF THUMB

An atomic beats a mutex when the fight over it is light. Under heavy contention that advantage evaporates — then you stop sharing the counter instead of picking a cleverer instruction.

Three ways to count to 160 million

Same arithmetic, same threads. Only *where in memory* the counters live changes.

```
// A one counter, shared by everyone
std::atomic<int64_t> counter{0};

// B one counter per thread, packed
struct Packed { std::atomic<int64_t> v; };
std::vector<Packed> c(threads);

// C same, one cache line each
struct alignas(64) Padded {
    std::atomic<int64_t> v; };
```

All three increment with `memory_order_relaxed`: a counter only needs its updates not to be lost. Publishing *data* needs more than that — in the memory-model block later today.

? QUESTION FOR YOU

Same work, same sixteen threads. Rank A, B and C by wall time — and guess the gap between them.

Three ways to count to 160 million

Same arithmetic, same threads. Only *where in memory* the counters live changes.

```
// A one counter, shared by everyone
std::atomic<int64_t> counter{0};

// B one counter per thread, packed
struct Packed { std::atomic<int64_t> v; };
std::vector<Packed> c(threads);

// C same, one cache line each
struct alignas(64) Padded {
    std::atomic<int64_t> v; };
```

All three increment with `memory_order_relaxed`: a counter only needs its updates not to be lost. Publishing *data* needs more than that — in the memory-model block later today.

What it printed

```
$ ./count3
160000000 increments, 16 threads
```

A one shared atomic	3.47 s
B per-thread, packed	0.99 s
C per-thread, alignas(64)	0.06 s

Median of 7 runs. i7-14700HX, 20 cores, GCC 16.2, -O2.

B is the obvious fix, and it does buy you 3× — so you stop there. It leaves 17× on the table: eight counters still sit on one 64-byte line. The hardware moves *lines*, not variables.

■ RULE OF THUMB

Nearly everyone puts B next to C, because the counters are already private. Thief 2 is back: one keyword, `alignas(64)`, beats any clever instruction you could have reached for.

CAS: the loop under every lock-free structure

fetch_add can only add. Want a running maximum, or an add that stops at a cap? Then you read, compute, and write back *only if* nobody beat you.

Compare-and-swap, in full

```
int old = v.load();
while (!v.compare_exchange_weak(
    old, f(old)))
    ; // spurious, or someone beat us; old refreshed
```

“Write the new value, but only if the old one is still there.” So “atomic” never meant “one instruction”: this is a loop that retries until it wins.

Missed it? Your read is refreshed for free — go around again. weak may also fail for no reason at all: that is why it lives in a loop.

Why this matters

Every lock-free stack, queue and counter you will ever read is this loop with a different `f`.

It is also the fast path of a mutex: taking an uncontended lock is one successful CAS — which is exactly where we go next.

ABA: the value went $A \rightarrow B \rightarrow A$ behind your back, and CAS compares values, not history. Harmless for a counter; fatal when the value is a pointer to a freed and reused node — then swap a version counter along with it.

? QUESTION FOR YOU

Your CAS returned true. Does that mean nobody touched the cell while you were computing?

CAS: the loop under every lock-free structure

fetch_add can only add. Want a running maximum, or an add that stops at a cap? Then you read, compute, and write back *only if* nobody beat you.

Compare-and-swap, in full

```
int old = v.load();
while (!v.compare_exchange_weak(
    old, f(old)))
    ; // spurious, or someone beat us; old refreshed
```

“Write the new value, but only if the old one is still there.” So “atomic” never meant “one instruction”: this is a loop that retries until it wins.

Missed it? Your read is refreshed for free — go around again. weak may also fail for no reason at all: that is why it lives in a loop.

Why this matters

Every lock-free stack, queue and counter you will ever read is this loop with a different `f`.

It is also the fast path of a mutex: taking an uncontended lock is one successful CAS — which is exactly where we go next.

ABA: the value went $A \rightarrow B \rightarrow A$ behind your back, and CAS compares values, not history. Harmless for a counter; fatal when the value is a pointer to a freed and reused node — then swap a version counter along with it.

? QUESTION FOR YOU

Your CAS returned true. Does that mean nobody touched the cell while you were computing?

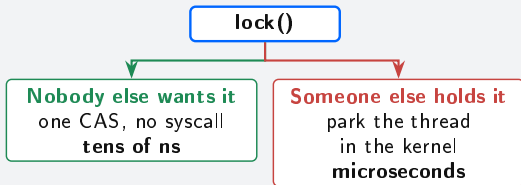
→ ANSWER

No. It means the value you see now equals the value you saw then. Two threads could have taken it away and put it back.

What a mutex really costs

Everything depends on one thing: is anybody else holding it? The gap is two orders of magnitude.

One `lock()`, two very different stories



Tens of nanoseconds against microseconds. The lock itself is one atomic; the bill is a thread that goes to sleep — and the wake-up on the other

Falling asleep is the hard part

On Linux the slow path is a **futex** — a *fast user space mutex*. The lock word lives in your memory; the kernel is called only to park and to wake.

The tricky moment is the gap between “the lock looks busy” and “I am asleep”. If the owner unlocks inside it, nobody wakes you — so the kernel **re-checks the word** before parking, and refuses to sleep if it changed.

■ RULE OF THUMB

In user space, take the plain `mutex`. Its fast path is already one atomic instruction — you will not hand-write anything cheaper, and the contended case is the kernel’s job, not yours.

Spin or sleep?

Parking a thread costs microseconds. So why not keep the CPU and spin until the lock is free?

Spin only when *all* of these hold

- The section is **shorter than a context switch**: a few instructions, no I/O, no allocation.
- The owner **cannot be preempted** — kernel spinlocks disable preemption; pinned threads with no oversubscription come close.
- You cannot sleep at all: interrupt handlers.

A short section is *necessary*, never sufficient — point two decides. Hence kernels spin and your application does not.

Lock-holder preemption

More threads than cores, and the scheduler evicts exactly the thread that **holds** the lock. It is now not running at all.

Everybody else keeps spinning — burning whole time slices waiting for a thread that is not even on a core.

Many mutexes (Java monitors, adaptive pthread mutexes) spin briefly before parking; glibc's default `std::mutex` does not — it parks at once. Either way, with nobody contending the lock is already one CAS, so a spinlock wins you nothing.

? QUESTION FOR YOU

Your critical section is twenty nanoseconds long. Spinlock or mutex?

Spin or sleep?

Parking a thread costs microseconds. So why not keep the CPU and spin until the lock is free?

Spin only when *all* of these hold

- The section is **shorter than a context switch**: a few instructions, no I/O, no allocation.
- The owner **cannot be preempted** — kernel spinlocks disable preemption; pinned threads with no oversubscription come close.
- You cannot sleep at all: interrupt handlers.

A short section is *necessary*, never sufficient — point two decides. Hence kernels spin and your application does not.

Lock-holder preemption

More threads than cores, and the scheduler evicts exactly the thread that **holds** the lock. It is now not running at all.

Everybody else keeps spinning — burning whole time slices waiting for a thread that is not even on a core.

Many mutexes (Java monitors, adaptive pthread mutexes) spin briefly before parking; glibc's default `std::mutex` does not — it parks at once. Either way, with nobody contending the lock is already one CAS, so a spinlock wins you nothing.

? QUESTION FOR YOU

Your critical section is twenty nanoseconds long. Spinlock or mutex?

→ ANSWER

Mutex — unless you own the whole machine. The length of the section is not the deciding factor; whether the owner can be *preempted* is.

Who owns the lock?

A mutex has an owner — that decides what happens when the same thread asks twice.

Asking twice: two answers

```
synchronized void a() { b(); } // fine
synchronized void b() { ... } // same owner, count++
```

```
std::mutex m;
m.lock(); m.lock(); // undefined behavior
```

```
std::lock_guard<std::mutex> g(m); // the normal way
```

Java: a monitor is *reentrant* — it knows its owner and bumps a counter.

C++: relocking is UB. Let the guard unlock, on every path.

synchronized or ReentrantLock?

Both are reentrant and both are fast. Reach past synchronized only for what it cannot express:

- tryLock() — give up instead of waiting;
- tryLock(t) — wait with a timeout;
- lockInterruptibly() — cancel a waiter;
- new ReentrantLock(true) — fair mode;
- several Conditions on one lock.

The price: you now own the unlock(). synchronized never leaks a lock; ReentrantLock without finally does.

? QUESTION FOR YOU

A synchronized method calls another synchronized method of the same object. Does it hang?

Who owns the lock?

A mutex has an owner — that decides what happens when the same thread asks twice.

Asking twice: two answers

```
synchronized void a() { b(); } // fine
synchronized void b() { ... } // same owner, count++
```

```
std::mutex m;
m.lock(); m.lock(); // undefined behavior
```

```
std::lock_guard<std::mutex> g(m); // the normal way
```

Java: a monitor is *reentrant* — it knows its owner and bumps a counter.

C++: relocking is UB. Let the guard unlock, on every path.

synchronized or ReentrantLock?

Both are reentrant and both are fast. Reach past `synchronized` only for what it cannot express:

- `tryLock()` — give up instead of waiting;
- `tryLock(t)` — wait with a timeout;
- `lockInterruptibly()` — cancel a waiter;
- `new ReentrantLock(true)` — fair mode;
- several Conditions on one lock.

The price: you now own the `unlock()`. `synchronized` never leaks a lock; `ReentrantLock` without `finally` does.

? QUESTION FOR YOU

A `synchronized` method calls another `synchronized` method of the same object. Does it hang?

→ ANSWER

In Java, no — same owner, the counter just goes up. Write the same thing with a plain `std::mutex` and you are in **undefined behavior**. If a recursive mutex looks like the fix, the real problem is usually that your locking and your call graph disagree.

How coarse should your lock be?

One lock around the whole map makes every thread walk through the same door, one at a time.

Step 0: look at what is inside

```
synchronized void put(K k, String raw) {  
    log.info("put " + k);    // I/O  
    V val = parse(load(raw)); // allocation  
    map.put(k, val);         // the real work  
}
```

Same lock, one line inside

```
log.info("put " + k);    // outside  
V val = parse(load(raw)); // outside  
synchronized (this) { map.put(k, val); }
```

Step 1: split the lock

```
Lock b = lock[Math.floorMod(k.hashCode(), N)];  
b.lock();  
try { ... } finally { b.unlock(); }
```

What it costs you:

- anything spanning buckets (resize, size()) takes several locks
→ order them, or deadlock;
- size() and iteration stop being exact;
- more memory, much trickier code.

And usually you should not write it at all: ConcurrentHashMap already did.

? QUESTION FOR YOU

The service crawls. The profiler says 80 % of the time is spent waiting on one mutex. What do you do first?

How coarse should your lock be?

One lock around the whole map makes every thread walk through the same door, one at a time.

Step 0: look at what is inside

```
synchronized void put(K k, String raw) {  
    log.info("put " + k);           // I/O  
    V val = parse(load(raw));       // allocation  
    map.put(k, val);                // the real work  
}
```

Same lock, one line inside

```
log.info("put " + k);               // outside  
V val = parse(load(raw));           // outside  
synchronized (this) { map.put(k, val); }
```

Step 1: split the lock

```
Lock b = lock[Math.floorMod(k.hashCode(), N)];  
b.lock();  
try { ... } finally { b.unlock(); }
```

What it costs you:

- anything spanning buckets (resize, size()) takes several locks → order them, or deadlock;
- size() and iteration stop being exact;
- more memory, much trickier code.

And usually you should not write it at all: ConcurrentHashMap already did.

? QUESTION FOR YOU

The service crawls. The profiler says 80 % of the time is spent waiting on one mutex. What do you do first?

→ ANSWER

Shrink the section before you split the lock. What is serialized is the code *inside* it — the other cores keep doing unrelated work. Push I/O, allocation and logging out, and the queue often disappears on its own.

Deadlock needs all four Coffman conditions

A deadlock happens only when **all four** hold at once — so break any one of them. We will come back to these four when the philosophers sit down to dinner.

The classic two-mutex trap

```
A: m1.lock();   B: m2.lock();  
A: m2.lock();   // waits for B  
B: m1.lock();   // waits for A
```

The four Coffman conditions:

1. **Mutual exclusion** — while one thread holds it, nobody else can.
2. **Hold and wait** — I keep one lock while asking for another.
3. **No preemption** — nobody can take the lock away.
4. **Circular wait** — the wait-for graph closes a loop.

Two fixes, two different conditions

1. **Kill the circular wait.** Every thread takes the locks in one fixed order (say, by address) — then no loop can form.

```
std::lock_guard a(m1); // same order everywhere  
std::lock_guard b(m2);
```

2. **Break hold and wait.** Take them together or not at all: grab the first, try_lock the rest, and on failure drop *everything* and retry. Hand-rolled, back off a *random* amount — or you trade deadlock for livelock.

```
std::scoped_lock lk(m1, m2); // try, back off, retry
```

? QUESTION FOR YOU

Thread A takes m1 then m2; thread B takes them the other way round. Does the program always hang?

Deadlock needs all four Coffman conditions

A deadlock happens only when **all four** hold at once — so break any one of them. We will come back to these four when the philosophers sit down to dinner.

The classic two-mutex trap

```
A: m1.lock();   B: m2.lock();  
A: m2.lock();   // waits for B  
B: m1.lock();   // waits for A
```

The four Coffman conditions:

1. **Mutual exclusion** — while one thread holds it, nobody else can.
2. **Hold and wait** — I keep one lock while asking for another.
3. **No preemption** — nobody can take the lock away.
4. **Circular wait** — the wait-for graph closes a loop.

Two fixes, two different conditions

1. **Kill the circular wait.** Every thread takes the locks in one fixed order (say, by address) — then no loop can form.

```
std::lock_guard a(m1); // same order everywhere  
std::lock_guard b(m2);
```

2. **Break hold and wait.** Take them together or not at all: grab the first, try_lock the rest, and on failure drop *everything* and retry. Hand-rolled, back off a *random* amount — or you trade deadlock for livelock.

```
std::scoped_lock lk(m1, m2); // try, back off, retry
```

? QUESTION FOR YOU

Thread A takes m1 then m2; thread B takes them the other way round. Does the program always hang?

→ ANSWER

No — and that's the **worst** part: your local tests stay green, and then it freezes solid under production load.

Happens-before: the one memory-model rule you need

Compiler and CPU reorder freely — as long as a *single* thread cannot tell.

The one rule to keep

If *A happens-before B*, then every write made before *A* is **guaranteed visible** at *B*.



B reads 42 — guaranteed by that edge, not by timing.

Where you get an edge for free

- `unlock(m)` → the next `lock(m)` on the same mutex;
- `start()` → the first line of the new thread;
- the thread finishes → `join()` returns;
- a task is submitted → the pool runs it;
- a flag written and read the right way — *next slide*.

No edge, no guarantee

With no edge the reader may return a **stale value forever**, and nothing forbids it.

? QUESTION FOR YOU

One thread spins in `while (!stop)`; another writes `stop = true`, and `stop` is a plain `bool`. Does the loop ever exit?

Happens-before: the one memory-model rule you need

Compiler and CPU reorder freely — as long as a *single* thread cannot tell.

The one rule to keep

If *A happens-before B*, then every write made before *A* is **guaranteed visible** at *B*.



B reads 42 — guaranteed by that edge, not by timing.

Where you get an edge for free

- `unlock(m)` → the next `lock(m)` on the same mutex;
- `start()` → the first line of the new thread;
- the thread finishes → `join()` returns;
- a task is submitted → the pool runs it;
- a flag written and read the right way — *next slide*.

No edge, no guarantee

With no edge the reader may return a **stale value forever**, and nothing forbids it.

? QUESTION FOR YOU

One thread spins in `while (!stop)`; another writes `stop = true`, and `stop` is a plain `bool`. Does the loop ever exit?

→ ANSWER

No edge, so the compiler may hoist the read out of the loop and leave you with `while (true)`. In C/C++ that is a data race and therefore **undefined behavior**; a plain Java field gives you no visibility either.

Publishing data with a flag

The most common pattern in real code: fill a structure, then flip a flag.

Java: one keyword is enough

```
int[] data;
volatile boolean ready;
```

```
data = compute();           // writer
ready = true;                // release
```

```
if (ready) use(data);        // acquire: data visible
```

The volatile write publishes everything written before it, and the volatile read on the other side picks it all up.

C++: the same keyword does not work

```
// volatile bool ready; useless here
std::atomic<bool> ready{false}; // use this
```

```
data = compute();           // writer
ready.store(true, std::memory_order_release);
```

```
if (ready.load(std::memory_order_acquire)) use(data);
```

C++ volatile means only “do not optimize this access away”. It orders nothing: neither compiler nor CPU has to keep data before ready, and the race on data is still UB. Plain store/load default to seq_cst — also correct, just slower.

! DON'T DO THIS

Java volatile buys **visibility**, not **atomicity**: a volatile counter still loses increments, because `x++` is read-modify-write.

And the debt from the demo: its counters may use relaxed, because nobody reads data *through* them and the total is read after `join()` — that is the edge. A flag that publishes data has no such luck: without the release/acquire edge the reader **may** see the flag and stale data.

Double-checked locking: correct-looking, broken

Create it once, then never pay for the lock again — and race where it looks safest.

The classic broken version

```
static Config cfg;           // not volatile
static Config get() {
    if (cfg == null) {       // 1st check, no lock
        synchronized (Config.class) {
            if (cfg == null)  // 2nd check, under lock
                cfg = new Config(); // too early
        }
    }
    return cfg;
}
```

Where the race hides

`new Config()` is not one step: allocate, run the constructor, store the reference. Nothing forces that order.

A second thread takes the first check — no lock, no edge — and can see a non-null reference to a half-built object.

Four fixes

- Java: make the field `volatile` — the edge is back.
- Java, simpler: the *holder* idiom, a static nested class the JVM initializes lazily.
- Java, best: make every field `final` — a racing reader then sees null or a finished object, never a half-built one.

? QUESTION FOR YOU

The object is created **under the lock**. So where is the race?

Double-checked locking: correct-looking, broken

Create it once, then never pay for the lock again — and race where it looks safest.

The classic broken version

```
static Config cfg;           // not volatile
static Config get() {
    if (cfg == null) {       // 1st check, no lock
        synchronized (Config.class) {
            if (cfg == null)  // 2nd check, under lock
                cfg = new Config(); // too early
        }
    }
    return cfg;
}
```

Where the race hides

`new Config()` is not one step: allocate, run the constructor, store the reference. Nothing forces that order.

A second thread takes the first check — no lock, no edge — and can see a non-null reference to a half-built object.

Four fixes

- Java: make the field `volatile` — the edge is back.
- Java, simpler: the *holder* idiom, a static nested class the JVM initializes lazily.
- Java, best: make every field `final` — a racing reader then sees null or a finished object, never a half-built one.

? QUESTION FOR YOU

The object is created **under the lock**. So where is the race?

→ ANSWER

Not in the writer — in the **reader that skipped the lock**. Its first check has no edge to the construction, so it may see the reference before the fields.

Condition variables: waiting without burning CPU

A mutex protects data. A condition variable answers a different question: how do you *wait for something to become true* without spinning on it?

C++: the lock and the condvar are separate

```
std::unique_lock lk(m);  
while (q.empty())      // while, never if  
    cv.wait(lk);       // drops m, sleeps, retakes m  
auto item = q.front(); q.pop();  
  
cv.wait(lk, []{ return !q.empty(); }); // same loop,  
    idiomatic
```

`wait()` releases the mutex and goes to sleep as one atomic step, so no notification can slip through the gap. You wake up holding the mutex again.

Java: every object is its own monitor

```
synchronized (q) {          // throws  
    InterruptedException  
    while (q.isEmpty())  
        q.wait();           // releases the monitor of q  
    item = q.remove();  
}  
synchronized (q) { q.add(x); q.notifyAll(); } // producer
```

The mutex is hidden *inside the object*: no separate condvar to pass around. `wait()` is legal only while you hold that object's monitor — otherwise `IllegalMonitorStateException`.

? QUESTION FOR YOU

Swap that `while` for a plain `if`. What breaks?

Condition variables: waiting without burning CPU

A mutex protects data. A condition variable answers a different question: how do you *wait for something to become true* without spinning on it?

C++: the lock and the condvar are separate

```
std::unique_lock lk(m);  
while (q.empty())      // while, never if  
    cv.wait(lk);       // drops m, sleeps, retakes m  
auto item = q.front(); q.pop();  
  
cv.wait(lk, []{ return !q.empty(); }); // same loop,  
    idiomatic
```

`wait()` releases the mutex and goes to sleep as one atomic step, so no notification can slip through the gap. You wake up holding the mutex again.

Java: every object is its own monitor

```
synchronized (q) {           // throws  
    InterruptedException  
    while (q.isEmpty())  
        q.wait();           // releases the monitor of q  
    item = q.remove();  
}  
synchronized (q) { q.add(x); q.notifyAll(); } // producer
```

The mutex is hidden *inside the object*: no separate condvar to pass around. `wait()` is legal only while you hold that object's monitor — otherwise `IllegalMonitorStateException`.

? QUESTION FOR YOU

Swap that `while` for a plain `if`. What breaks?

→ ANSWER

You pop an empty queue. A wakeup does not promise the condition holds: it can be spurious, or another consumer took the item first. The loop re-checks; an `if` just trusts it.

Lost wakeup: signal or broadcast?

A notification sent while nobody is waiting simply evaporates: a condition variable has no memory and counts nothing.

The signal nobody heard



How, if `wait()` is atomic? B tested `q.empty()` *outside* the lock — that is the gap. Test the predicate, change it and signal under the *same* mutex, and the atomic release closes it.

signal vs broadcast

- `signal()` wakes **at least one** waiter — POSIX allows more, one more reason never to skip the loop. `broadcast()` wakes them all. (Java: `notify()` picks one arbitrary waiter, `notifyAll()` everyone.)
- **The signal trap:** if threads wait on *different* conditions through one condvar, you may wake the wrong one. It sleeps again, the right one never wakes. **Everything hangs.**
- **The broadcast cost:** the whole crowd wakes and fights for one lock (*thundering herd*).

■ RULE OF THUMB

Wake one when *one* waiter can make progress; broadcast when several can, or when you cannot tell which. If the waiters want different things, give each condition its own condvar. And note what a condvar is not: a semaphore *does* remember — a `release()` with nobody waiting is not lost. That is the next slide.

A semaphore is a permit with no owner

A mutex asks “is it free?”. A semaphore asks “how many are still allowed in?” — and that one extra number changes what the primitive can promise you.

A counter of permits

```
std::counting_semaphore<> sem{10}; // C++20
sem.acquire(); // 10 -> 9, blocks at 0
try { callExternalApi(); }
catch (...) { sem.release(); throw; } // no RAI guard in
std
sem.release(); // 9 -> 10

sem.acquire();
try { call(); } finally { sem.release(); }
```

What it is for: capping concurrency — ten in-flight calls to an external API, a fixed-size buffer, a pool of N connections.

Nobody owns a permit

A mutex has an owner: whoever locked it is the one who unlocks it.

A semaphore does not. Thread A can acquire() and thread B can release() — that compiles, and sometimes it is what you want.

What goes away with the owner: recursion is meaningless, and priority inheritance has nobody to boost — it belongs to an owned lock, not to a counter.

? QUESTION FOR YOU

A semaphore initialized to 1. Is that just a mutex?

A semaphore is a permit with no owner

A mutex asks “is it free?”. A semaphore asks “how many are still allowed in?” — and that one extra number changes what the primitive can promise you.

A counter of permits

```
std::counting_semaphore<> sem{10}; // C++20
sem.acquire(); // 10 -> 9, blocks at 0
try { callExternalApi(); }
catch (...) { sem.release(); throw; } // no RAI guard in
std
sem.release(); // 9 -> 10

sem.acquire();
try { call(); } finally { sem.release(); }
```

What it is for: capping concurrency — ten in-flight calls to an external API, a fixed-size buffer, a pool of N connections.

Nobody owns a permit

A mutex has an owner: whoever locked it is the one who unlocks it.

A semaphore does not. Thread A can acquire() and thread B can release() — that compiles, and sometimes it is what you want.

What goes away with the owner: recursion is meaningless, and priority inheritance has nobody to boost — it belongs to an owned lock, not to a counter.

? QUESTION FOR YOU

A semaphore initialized to 1. Is that just a mutex?

→ ANSWER

It gives you mutual exclusion, but not a mutex: no owner, so no recursion, no priority inheritance, and any thread can release it. Say “one at a time” with a mutex, “at most N ” with a semaphore.

Barriers and latches: waiting for everybody

Here you wait not for a value, but for *the other threads*.

Latch: one-shot

```
CountDownLatch ready = new CountDownLatch(N);  
ready.countDown(); // each worker, once  
ready.await();     // main waits for 0
```

Counts to zero and stays there: no rearming.

Barrier: cyclic

```
CyclicBarrier step = new CyclicBarrier(N);  
step.await(); // resets each round
```

A barrier is your span, made by hand

Every round ends when the slowest thread arrives. Fifteen threads finish in 2 ms, one takes 20 ms — the round costs 20 ms, and you paid for sixteen.

That is the span from the previous lecture, except now *you* put it there: parallelism drains toward one thread as the others arrive and idle.

C++20 has the same pair: `std::latch` and `std::barrier`.

So: balance the work per round, or drop the barrier where the algorithm lets threads run ahead.

? QUESTION FOR YOU

You hand-roll a barrier on a counter and a condvar. The fast thread finishes the round, loops, and comes back *before* the slow one has woken up. What happens?

Barriers and latches: waiting for everybody

Here you wait not for a value, but for *the other threads*.

Latch: one-shot

```
CountDownLatch ready = new CountDownLatch(N);  
ready.countDown(); // each worker, once  
ready.await();    // main waits for 0
```

Counts to zero and stays there: no rearming.

Barrier: cyclic

```
CyclicBarrier step = new CyclicBarrier(N);  
step.await(); // resets each round
```

A barrier is your span, made by hand

Every round ends when the slowest thread arrives. Fifteen threads finish in 2 ms, one takes 20 ms — the round costs 20 ms, and you paid for sixteen.

That is the span from the previous lecture, except now *you* put it there: parallelism drains toward one thread as the others arrive and idle.

C++20 has the same pair: `std::latch` and `std::barrier`.

So: balance the work per round, or drop the barrier where the algorithm lets threads run ahead.

? QUESTION FOR YOU

You hand-roll a barrier on a counter and a condvar. The fast thread finishes the round, loops, and comes back *before* the slow one has woken up. What happens?

→ ANSWER

The **slow** thread is the victim: it wakes, re-checks the counter — and the fast one has already bumped it for the *next* round, so it sleeps again and misses its own release. That is why a real barrier counts **generations**: you wait while `(gen == my_gen)`, not for the counter to hit zero.

A bounded queue: putting it all together

The mutex and the condvars from this block in one object: one lock for the data, two condition variables for the two ways it can make you wait.

Producer and consumer, bounded

```
std::condition_variable not_full, not_empty;
void put(T x) {
    std::unique_lock lk(m);
    while (q.size() == cap) not_full.wait(lk);
    q.push(std::move(x));
    lk.unlock(); not_empty.notify_one();
}
// take() mirrors it -- and keeps the loop:
// while (q.empty()) not_empty.wait(lk);
// pop; then notify not_full.
```

Why two condvars, not one

Producers and consumers wait for **opposite** conditions. Put them on one condvar and `notify_one()` can wake a producer when you needed a consumer: it re-checks, sees the queue still full, sleeps again — and nobody moves.

One condvar forces `notify_all()` and a thundering herd. Two let you wake exactly the side that can make progress.

And cap is the point: a slow consumer pushes back on the producer instead of letting the queue eat your memory.

? QUESTION FOR YOU

Twenty lines, a lock, two condvars — and two ways to hang it. Was that a good use of your afternoon?

A bounded queue: putting it all together

The mutex and the condvars from this block in one object: one lock for the data, two condition variables for the two ways it can make you wait.

Producer and consumer, bounded

```
std::condition_variable not_full, not_empty;
void put(T x) {
    std::unique_lock lk(m);
    while (q.size() == cap) not_full.wait(lk);
    q.push(std::move(x));
    lk.unlock(); not_empty.notify_one();
}
// take() mirrors it -- and keeps the loop:
// while (q.empty()) not_empty.wait(lk);
// pop; then notify not_full.
```

Why two condvars, not one

Producers and consumers wait for **opposite** conditions. Put them on one condvar and `notify_one()` can wake a producer when you needed a consumer: it re-checks, sees the queue still full, sleeps again — and nobody moves.

One condvar forces `notify_all()` and a thundering herd. Two let you wake exactly the side that can make progress.

And cap is the point: a slow consumer pushes back on the producer instead of letting the queue eat your memory.

? QUESTION FOR YOU

Twenty lines, a lock, two condvars — and two ways to hang it. Was that a good use of your afternoon?

→ ANSWER

In Java, what you just wrote is called `ArrayBlockingQueue` and it ships with the JDK — write it once to understand it, then take the ready-made one. In C++ there is no blocking queue in the standard library: TBB, folly, or this code.

Four ways a lock can ruin your day

Before we compare locks, we need the words. These four failures look nothing alike — and the CPU meter already tells you which one you have.

Deadlock

Everybody stopped. Each waits for a lock another one holds, and the wait closes a loop.

On the meter: CPU near zero, requests hang forever.

Livelock

Everybody busy, nobody moves. Threads keep politely backing off and retrying — forever.

On the meter: CPU pinned at 100 %, throughput zero.

Starvation

The system is fine; *this* thread is not. Somebody is always ahead of it, so its turn never comes.

On the meter: healthy average, ugly p99.

Convoy

One slow holder makes everyone queue — and once the queue exists, every hand-off costs a wake-up. The convoy **outlives** the slow holder.

On the meter: N threads, the throughput of one.

? QUESTION FOR YOU

The service stopped answering and the CPU sits at 100 %. Is that a deadlock?

Four ways a lock can ruin your day

Before we compare locks, we need the words. These four failures look nothing alike — and the CPU meter already tells you which one you have.

Deadlock

Everybody stopped. Each waits for a lock another one holds, and the wait closes a loop.

On the meter: CPU near zero, requests hang forever.

Livelock

Everybody busy, nobody moves. Threads keep politely backing off and retrying — forever.

On the meter: CPU pinned at 100 %, throughput zero.

Starvation

The system is fine; *this* thread is not. Somebody is always ahead of it, so its turn never comes.

On the meter: healthy average, ugly p99.

Convoy

One slow holder makes everyone queue — and once the queue exists, every hand-off costs a wake-up. The convoy **outlives** the slow holder.

On the meter: N threads, the throughput of one.

? QUESTION FOR YOU

The service stopped answering and the CPU sits at 100 %. Is that a deadlock?

→ ANSWER

Almost certainly not: a deadlock on *blocking* locks **sleeps**. A full CPU with no progress is a livelock, a spin loop — or a deadlock on spinlocks. Check the CPU meter *before* the stacks: it halves your search.

Fair locks: no starvation, less throughput

A strict queue saves you from starvation, and it costs you throughput.

Why an unfair lock runs faster

A thread **already running on a core** grabs the free lock on the spot, instead of waiting for a sleeper to be woken. That's *barging*.

A fair lock must hand it to whoever is first in line — and waking that thread takes microseconds.

The price of barging: a queued thread can be skipped over again and again — the starvation we just named.

What you actually get to pick

- **Java** `ReentrantLock(true)` is the fair mode: *“lower overall throughput, often much slower”*. It orders *the queue* only — an untimed `tryLock()` still barges past it by design.
- **C++** `std::mutex` makes **no fairness promise** — never rely on the order threads get it.
- Many runtimes are **adaptive**: barging first, strict order after a long wait.

? QUESTION FOR YOU

You have a p99 latency SLA. Fair lock or unfair?

Fair locks: no starvation, less throughput

A strict queue saves you from starvation, and it costs you throughput.

Why an unfair lock runs faster

A thread **already running on a core** grabs the free lock on the spot, instead of waiting for a sleeper to be woken. That's *barging*.

A fair lock must hand it to whoever is first in line — and waking that thread takes microseconds.

The price of barging: a queued thread can be skipped over again and again — the starvation we just named.

What you actually get to pick

- **Java** `ReentrantLock(true)` is the fair mode: *“lower overall throughput, often much slower”*. It orders *the queue* only — an untimed `tryLock()` still barges past it by design.
- **C++** `std::mutex` makes **no fairness promise** — never rely on the order threads get it.
- Many runtimes are **adaptive**: barging first, strict order after a long wait.

? QUESTION FOR YOU

You have a p99 latency SLA. Fair lock or unfair?

→ ANSWER

Still start unfair — it is faster on average. Switch to fair only when the **tail** breaks your SLA: watch p99/p999, never the average, because barging hides starvation there.

RW-lock: measure before you switch

Reads outnumber writes a hundred to one, readers never disturb each other — so a reader–writer lock must win. On short critical sections it usually does not.

A reader is not a free rider

“Many readers **or** one writer” is the guarantee. The implementation still has to *count* those readers.

So every reader does a **read-modify-write** on one shared counter — the same single cache line every other reader is hammering. Exactly what variant A of our demo was doing.

Your “parallel” readers serialize on the bookkeeping, not on the data.

Before you reach for one

- **It pays off when reads are long and frequent.** For a few-nanosecond lookup a plain mutex is usually ahead.
- **Writers can be starved:** a steady stream of readers keeps the writer waiting.
- **There is no upgrade.** Taking the write lock while still holding the read lock is **UB in C++**, a **hang in practice** — drop it and re-check.

■ RULE OF THUMB

Swapping mutex for an RW-lock is a **measurement**, not an optimization. Run both under your real read/write mix, and keep the one that wins.

Readers that never block

If counting readers is the bottleneck, stop counting them. Two designs let a reader touch no shared state at all — and both move the cost somewhere you have to plan for.

Seqlock: read twice, compare

A version counter. The writer bumps it by +1 when it starts and +1 when it is done, so an odd value means “a write is in flight”.

The reader **writes nothing**: it notes the version, reads the data, reads the version again — and retries if that version was **odd or changed**. Odd matters: start mid-write and both reads agree on torn data.

Readers are free; *writers* still take a mutex against each other.

RCU: the writer pays

The reader takes no lock: it just follows the pointer to the version that is published right now. Almost free — there is still some read-side bookkeeping.

The writer edits a **copy** and swaps the pointer atomically. The old copy may only be freed once every reader that could still see it is gone — the *grace period*.

This is how the Linux kernel reads many of its hot structures — routing tables, the dentry cache.

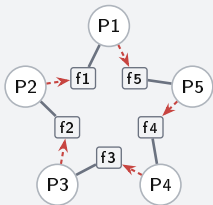
! DON'T DO THIS

As soon as a reader follows a **pointer**, the object has to outlive the read. Seqlock cannot protect a pointer at all; RCU can — the grace period *is* a **reclamation scheme**. Reach for a library that already implemented one.

Dining philosophers: deadlock-free is not starvation-free

Remember the four Coffman conditions? Numbering the forks breaks the circular wait — and the table still has a thinker who never eats. The puzzle is the gap between “nobody is stuck” and “everybody eats”.

All five grabbed their left fork



Grey: holds it. Red: waiting. The loop closes — circular wait, condition four.

Three fixes, three different promises

Fix	Deadlock	Starvation
Number the forks	gone	one may still never eat
A waiter hands out pairs	gone	gone — at the price of a central bottleneck
Chandy–Misra	gone	gone too

Every one of these promises assumes a fair scheduler underneath.

■ RULE OF THUMB

“It works” is not a guarantee. Name the one you actually need — no deadlock? no starvation? bounded wait? — and be ready to argue it.

Don't share at all

Everything so far was about sharing correctly. The cheapest lock in this whole lecture is the one you never take.

1. Never change it

Build the value once, hand out references freely. A read-only object needs no protection — there is no second writer to race with.

```
record Point(int x, int y) {}
```

Change means *new object*, not new value.

2. Give each thread its own

Confinement: the data never leaves one thread, so no other thread can reach it. Merge the per-thread results at the end.

```
thread_local Buf b; // C++
```

```
// Java
```

```
static final ThreadLocal<Buf> B  
    = ThreadLocal.withInitial(Buf::new);
```

Same idea as sharding in the demo. In a *pool* the value outlives the task — call `remove()`.

3. Copy, then publish

Copy-on-write: readers keep reading the old snapshot, the writer swaps in a finished new one.

```
atomic<shared_ptr<const Cfg>> cur;  
  
auto next = // copy  
    make_shared<Cfg>(*cur.load());  
next->add(x); // edit  
cur.store(move(next)); // publish
```

Cheap reads, expensive writes. The `shared_ptr` refcount *is* the reclamation; in Java the GC is.

! DON'T DO THIS

`const` in C++ promises you nothing about threads — a `const` method may still mutate a cached field behind your back. Java `final` does guarantee the field is safely published, but says nothing about the object it points at: a `final List` can still be modified by everybody.

Somebody already wrote your concurrent map

If you do have to share, the next question is not “which lock” — it is “has this already been written for me”.

Java: it is in the box

```
Map<K,Integer> m = new ConcurrentHashMap<>();  
m.merge(k, 1, Integer::sum); // atomic  
List<T> l = new CopyOnWriteArrayList<>();
```

The lock is sharded per bucket, reads mostly take none at all — the very thing we sketched by hand back on the granularity slide, only debugged by the JDK team.

C++: the standard has none

There is no concurrent container in the standard library. A `std::map` guarded by one mutex is *safe*, and it serializes every single lookup.

So you take a library — `oneTBB concurrent_hash_map`, `folly`, `libc++` — or you write it, and then you own every bug in it.

Writing your own is the *last* step of the ladder, not the first.

! DON'T DO THIS

A ready-made container makes **one call** atomic. It does not make **two calls** atomic: `containsKey` followed by `put` is the same check-then-act race condition we opened the lecture with — and no container can fix it for you. Use the one call that does both: `putIfAbsent`, `compute`, `merge`.

Stop synchronizing, start handing off

One level up the whole question changes: instead of guarding shared state, hand the work away and wait for a result.

A result you wait for

```
auto f = std::async(std::launch::async, // C++
                   work, arg);
int r = f.get(); // blocks until ready

var pool = Executors.newFixedThreadPool(8); // Java
Future<Integer> f = pool.submit(task);
```

No shared variable, no lock: the value crosses threads once, and the handoff carries the happens-before edge for you.

A pipeline you never block in

```
load(id) // CompletableFuture
    .thenApply(this::parse)
    .thenCombine(prices(), Order::of)
    .exceptionally(ex -> Order.EMPTY);
```

Each stage runs when its input is ready. Nobody holds a thread while waiting — which is the whole point of not blocking.

! DON'T DO THIS

Pool exhaustion. A task running in the pool submits another task to the *same* pool and blocks on its result. Every worker ends up waiting for work that has nowhere to run, and the service freezes solid. Worst part: no thread owns a monitor here, so jstack will **not** print “Found one Java-level deadlock” — you just see a pool of threads parked in get(). Rule: never block inside a pool on that same pool.

Races are caught by tools, not by reading

A race shows up in one run and hides in the next. “I ran it a hundred times and it was fine” proves nothing.

C/C++: ThreadSanitizer

It instruments every memory access and rebuilds the happens-before graph:

```
g++ -fsanitize=thread -g -O1 a.cpp
```

It reports the race **even when that run came out correct**. Costs $\approx 5\text{--}15\times$ slowdown.

Also: Helgrind flags risky mutex ordering.

JVM: Lincheck

It hammers your data structure with concurrent operations, and on failure hands you an **interleaving you can replay step by step**.

But TSan sees *data races* only. The containsKey-then-put race from the last block is invisible to it — no unsynchronized access. That one is Lincheck’s job.

Also: jctstress, for memory-model micro-tests.

■ RULE OF THUMB

A clean run is **evidence, not proof**: a detector only sees the accesses that actually executed this time. Run the tool on every path you care about, in CI, not once on your laptop.

A deadlock in production: how to actually see it

The service stops answering and the CPU is idle. You have one minute and a terminal — what do you type?

Java

```
jstack <pid>
Found one Java-level deadlock:
"worker-2" waiting to lock <0x7f..a8>
  which is held by "worker-1"
"worker-1" waiting to lock <0x7f..c0>
  which is held by "worker-2"
```

The JVM finds the cycle for you — but only for monitors and Lock objects it tracks.

C/C++

```
gdb -p <pid>
(gdb) thread apply all bt
```

Nobody draws the cycle for you here. Look for several threads parked in `__lll_lock_wait` and read the frame above it: that tells you which lock each one wants.

Then build the wait-for graph on paper — it is usually two edges long.

! DON'T DO THIS

Not every hang is a lock cycle, and `jstack` will not label this one. Look instead for N workers parked in `FutureTask.get` over an empty queue: the **pool itself** is the deadlock.

Which primitive do I actually need?

Read this table left to right: name the guarantee you need first, then take the simplest tool that gives it.

What you need	Primitive	What to watch out for
A counter, a flag, one value	<code>atomic</code> / CAS	Cheap alone; shard it under contention
An invariant across several fields	<code>mutex</code>	Keep the critical section short
Wait until something becomes true	condition variable	Always <code>while</code> ; keep the state in the data
Let only N of them in at once	semaphore	No owner: any thread may release a permit
Wait for N tasks, or for everybody	<code>latch</code> / <code>barrier</code>	A latch is one-shot; a barrier is reusable
A ready-made thread-safe collection	concurrent container	One call is atomic; a sequence of them is not
A result from another thread	<code>future</code> / <code>executor</code>	Never block in a pool on that same pool

? QUESTION FOR YOU

Name the primitive for each: **(1)** count processed requests across 16 threads; **(2)** never more than 10 concurrent calls to an external API; **(3)** warm the cache once, everybody else waits until it is ready.

Which primitive do I actually need?

Read this table left to right: name the guarantee you need first, then take the simplest tool that gives it.

What you need	Primitive	What to watch out for
A counter, a flag, one value	<code>atomic</code> / CAS	Cheap alone; shard it under contention
An invariant across several fields	<code>mutex</code>	Keep the critical section short
Wait until something becomes true	condition variable	Always <code>while</code> ; keep the state in the data
Let only N of them in at once	semaphore	No owner: any thread may release a permit
Wait for N tasks, or for everybody	<code>latch</code> / <code>barrier</code>	A latch is one-shot; a barrier is reusable
A ready-made thread-safe collection	concurrent container	One call is atomic; a sequence of them is not
A result from another thread	<code>future</code> / <code>executor</code>	Never block in a pool on that same pool

? QUESTION FOR YOU

Name the primitive for each: **(1)** count processed requests across 16 threads; **(2)** never more than 10 concurrent calls to an external API; **(3)** warm the cache once, everybody else waits until it is ready.

→ ANSWER

(1) an `atomic` — and a sharded one (`LongAdder`) the moment it gets hot. **(2)** a semaphore with 10 permits, not a `mutex`. **(3)** a latch (or `call_once` / the holder idiom) — one-shot is exactly the guarantee you want.

Four thieves, one checklist

You were promised $N\times$. Here is who took it, and what you do about each one.

THIEF 1

The algorithm

Amdahl caps you at $1/(1 - p)$; contention bends the curve back down.

Counter-move: plot the Karp-Flatt serial fraction e as N grows. One number tells you nothing — the *trend* names the culprit.

THIEF 2

The hardware

Cores are fast; memory is not. Two threads on one cache line fight over it.

Counter-move: pad hot per-thread data to its own line. Check bandwidth first.

THIEF 3

The OS

Every kernel thread costs a stack and a slot; a switch costs microseconds.

Counter-move: never a thread per task. Pool sized to the cores; async for waiting.

THIEF 4

Your own locks

Whatever sits inside the critical section never scales.

Counter-move: shrink it. Move I/O, allocation and logging out; shard before you get clever.

■ RULE OF THUMB

Start by **sharing nothing**. When you must share, pick the simplest primitive that gives you the guarantee you need — then **measure**, because every thief on this slide is invisible until you do.

Take-home sheet: the same primitive in C++ and Java

What you need	C++	Java	The trap
One value, atomically	<code>std::atomic<T></code>	<code>AtomicInteger</code> , <code>LongAdder</code>	Hot line: one shared counter serializes
Compare-and-set	<code>compare_exchange_weak</code>	<code>compareAndSet</code>	A retry loop, not one instruction; ABA
Plain mutual exclusion	<code>std::mutex</code> + <code>lock_guard</code>	<code>synchronized</code>	Re-locking <code>std::mutex</code> is UB; synchronized re-enters
Timeout, interrupt, fairness	<code>timed_mutex::try_lock_for</code> (no interrupt, no fair mode)	<code>ReentrantLock</code>	Fair mode trades throughput for order
Initialize once, lazily	<code>call_once</code> / static local	holder idiom, or DCL with <code>volatile</code>	DCL without <code>volatile</code> is broken
Visibility of a flag	<code>atomic</code> + <code>release/acquire</code>	<code>volatile</code>	C++ <code>volatile</code> is not for threads; Java <code>volatile x++</code> is not atomic
Spin instead of sleep	hand-written, rarely	—	Lock-holder preemption: the spinners burn their cores
Wait for a condition	<code>condition_variable</code>	<code>wait</code> / <code>notifyAll</code>	Always while; a lost signal is gone forever
Cap the concurrency	<code>counting_semaphore</code>	<code>Semaphore</code>	No owner, so no priority inheritance
Wait for <i>N</i> tasks	<code>latch</code>	<code>CountDownLatch</code>	One-shot: you cannot rearm it
Sync phase by phase	<code>barrier</code>	<code>CyclicBarrier</code>	All wait for the slowest; fixed party size
Many long reads	<code>shared_mutex</code>	<code>ReentrantReadWriteLock</code>	Short sections: often slower than a <code>mutex</code>
A concurrent map	TBB / folly (not in the STL)	<code>ConcurrentHashMap</code>	One call is atomic, two in a row are not
A blocking queue	TBB <code>concurrent_bounded_queue</code>	<code>ArrayBlockingQueue</code>	An <i>unbounded</i> queue replaces a hang with an OOM
Hand off work	<code>async</code> / <code>future</code>	<code>ExecutorService</code>	Waiting in the pool on the same pool deadlocks